🔹 1️⃣ Server-Side Session (Classic Stateful Auth)

Workflow / Life Cycle:

1. 📝 **Register/Login**
   - User login → password doğrulama
   - Server session açar → `req.session.userId`, `req.session.auth = true`
2. 🍪 **Session Storage**
   - Cookie ile client'e session id gönderilir (`connect.sid`)
3. 🔒 **Protected Route**
   - Client cookie gönderir → server session kontrol eder (`req.session.auth`)
   - Geçerli ise route çalışır, değilse 401
4. 🚪 **Logout**
   - Server session destroy edilir
   - Cookie client'de geçersiz olur
5. ⏰ **Expiration**
   - Session timeout'a göre otomatik destroy veya Redis/Memory cleanup

---

🔹 2️⃣ JWT (Stateless Token-Based Auth)

Workflow / Life Cycle:

1. 📝 **Login**
   - User login → password doğrulama
   - Server JWT üretir (payload: userId, iat, exp)
2. 💾 **Client Storage**
   - Token LocalStorage / Cookie / Secure Storage'da saklanır
3. 🔑 **Authenticated Request**
   - Client request → `Authorization: Bearer <token>` header ile gönderir
   - Server token verify eder (signature + expiration)
4. ⏰ **Token Expiration**
   - Token süresi dolunca invalid
   - Client refresh token ile yeni token alabilir (opsiyonel)
5. 🚫 **Logout / Revocation**
   - Stateless → token invalidate etmek zor (blacklist/DB gerek)

🔷 Karşılaştırmalı Özet Tablo

Özellik	Session (Stateful)	JWT (Stateless)
Storage	Server memory / Redis	Client (LocalStorage / Cookie)
Auth check	`req.session.auth`	`verify(token)`
Logout	`session.destroy()` → anında geçersiz	Token süresi dolana kadar geçerli
Scaling	Zor, stateful	Kolay, stateless
Browser uyumu	Çok iyi (cookie)	SPA / mobile / server-to-server ideal
Expiration	Session timeout	`exp` claim, refresh token opsiyonel
Revocation	Kolay	Zor (blacklist)